

APEXPLANET INTERNSHIP

TASK 5 — CAPSTONE PROJECT

APEXPLANET SOFTWARE PVT. LTD.

Domain: Cybersecurity and Ethical Hacking

**DVWA Web Application Penetration Testing
and Incident Response Simulation**

Submitted by :

NIMMALA NISHEK RATNA

Department: Computer Science and Engineering

Sathyabama Institute of Science and Technology

Date: 15 September 2026

1. Executive Summary

This capstone project demonstrates an end-to-end web application security testing and incident response workflow using Damn Vulnerable Web Application (DVWA) in an authorized local laboratory environment. The project applies practical cybersecurity skills across reconnaissance, service identification, HTTP analysis, controlled vulnerability testing, evidence collection, log-based detection, incident analysis, containment, eradication, and recovery.

A controlled SQL Injection condition was demonstrated in DVWA at the Low security level. A harmless request was then used to simulate an incident and verify detection in Apache access logs. The event was analyzed, temporarily contained using an iptables firewall rule, the temporary rule was removed, and DVWA availability was verified after recovery.

No real-world systems, external targets, real credentials, or production data were involved.

2. Project Overview

Item	Details
Organization	ApexPlanet
Internship	Cybersecurity and Ethical Hacking
Task	Task 5 — Capstone Project & Incident Response
Project	DVWA Web Application Penetration Testing and Incident Response Simulation
Target	Damn Vulnerable Web Application (DVWA)
Web Server	Apache2
Platform	Kali Linux
Scope	Authorized local virtual laboratory only

2.1 Objectives

- Assess an intentionally vulnerable web application.
- Perform reconnaissance and service identification.
- Analyze HTTP response headers and web-server activity.
- Demonstrate a controlled SQL Injection vulnerability.
- Collect and document evidence.
- Detect and analyze a simulated security event in Apache logs.
- Contain the simulated event using a firewall control.
- Remove the temporary control and verify recovery.
- Document findings, mitigations, and incident-response lessons learned.

2.2 Scope

In Scope

- DVWA on the local Kali Linux system.
- Apache2 web service and access logs.
- Nmap service identification.
- CURL HTTP header inspection.
- Controlled SQL Injection testing.

- Log-based incident detection and analysis.
- Temporary iptables containment.
- Recovery verification.

Out of Scope

- Public or third-party systems.
- Unauthorized accounts or data.
- Real credential collection.
- Real malware deployment.
- Disruption of external services.
- Persistence or destructive activity.

2.3 Tools and Technologies

Tool	Purpose
Kali Linux	Security testing platform
DVWA	Intentionally vulnerable web application
Apache2	Web server
Nmap	Reconnaissance/service identification
CURL	HTTP request/header inspection
Apache access logs	Event detection and analysis
iptables	Temporary containment
Web browser	Testing and recovery verification

2.4 Timeline

Phase	Activity
1	Planning and project selection
2	Reconnaissance and scanning
3	Controlled vulnerability testing and evidence
4	Incident detection, analysis, containment, eradication and recovery
5	Findings, mitigations and post-incident documentation
6	Final report, GitHub and presentation

3. Network / Architecture Diagram

The project uses a local controlled architecture in which the tester interacts with DVWA through Apache. Apache records requests in its access log, while iptables provides a temporary containment control during the incident-response simulation.

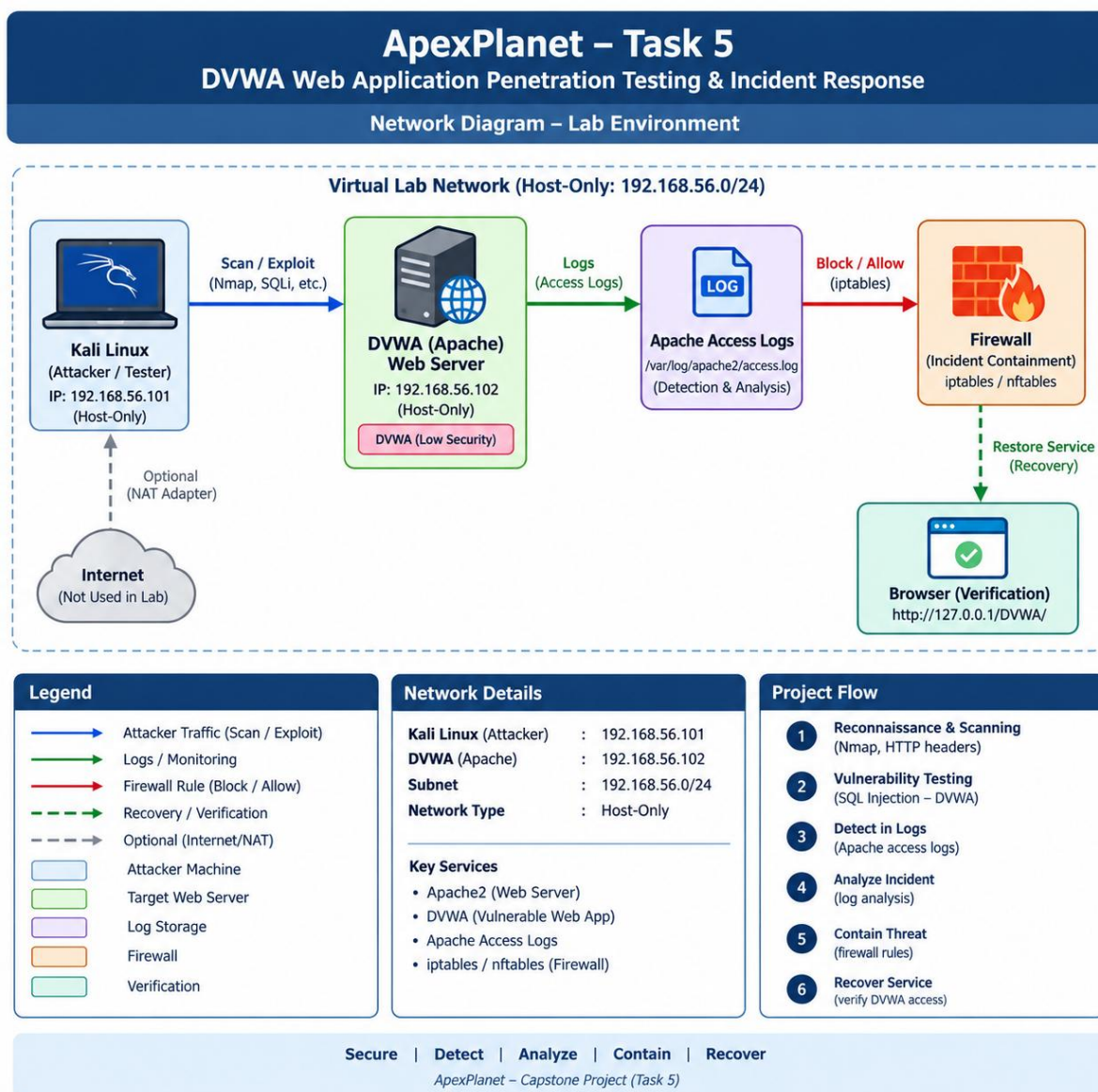


Figure 1. Task 5 Network / Architecture Diagram

4. Methodology

Testing followed a controlled methodology aligned with the Task 5 requirements. Each activity was limited to the authorized laboratory environment.

1. Reconnaissance and service identification.
2. HTTP response/header inspection.
3. Baseline Apache log review.
4. Controlled SQL Injection validation.
5. Evidence collection.
6. Harmless incident simulation.
7. Log-based detection and analysis.
8. Temporary firewall containment.
9. Removal of the temporary rule.

10. Recovery verification.

5. Security Testing and Evidence

5.1 DVWA Security Configuration

DVWA was operated at Low security to enable controlled demonstration of a known web-application weakness. This is appropriate for a deliberately vulnerable training application but is not a secure production configuration.

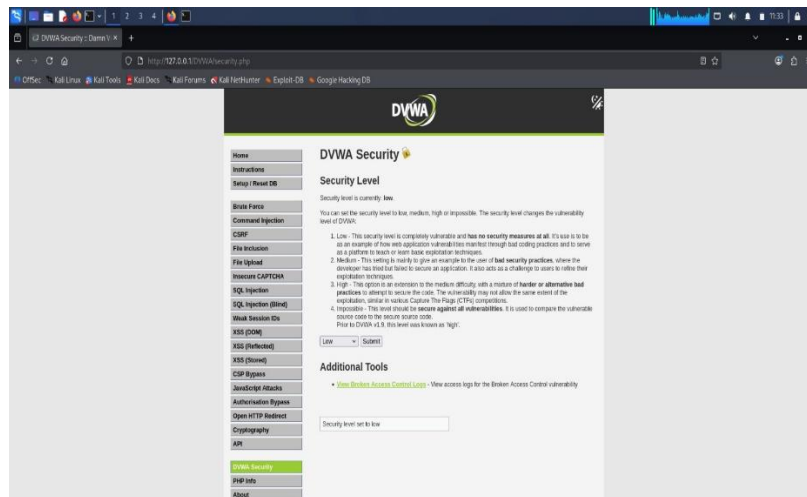


Figure 2.DVWA_Security_Level_Low

5.2 Reconnaissance and Scanning

Nmap was used against the local DVWA web service to identify the HTTP service and version information before application-level testing.

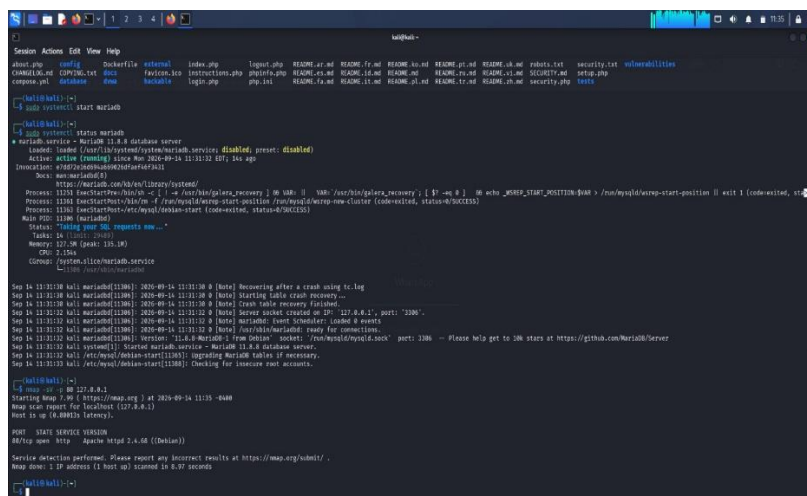


Figure 3.Nmap_DVWA

5.3 HTTP Header Analysis

CURL was used to inspect HTTP response headers returned by DVWA, providing an initial view of web-server response configuration and hardening opportunities.

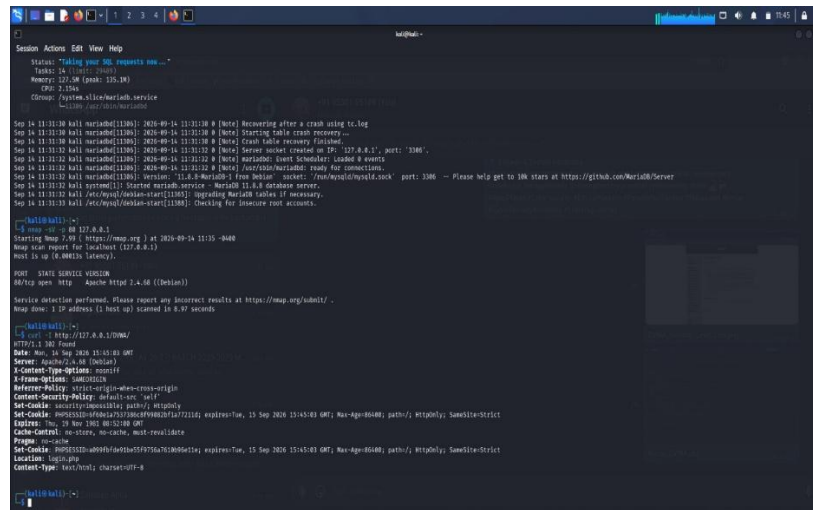


Figure 4.HTTP_Headers

5.4 Apache Access Log Baseline

The Apache access log was reviewed before the incident simulation to establish request visibility and confirm that web activity was being recorded.

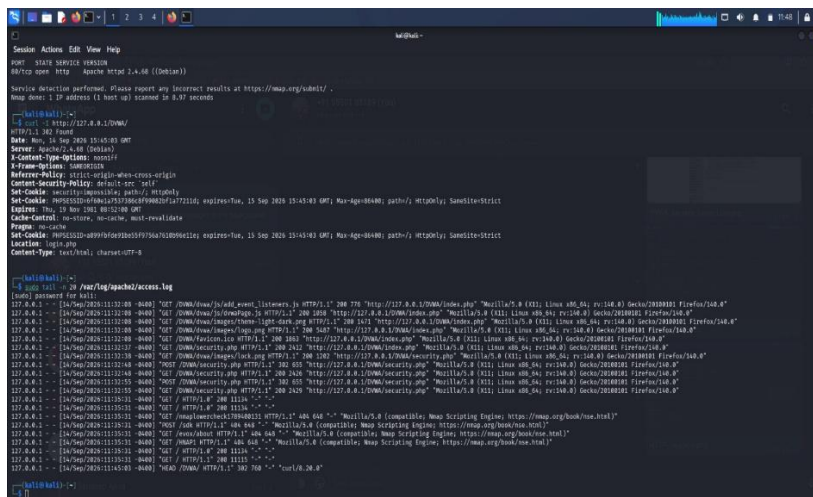


Figure 5. Apache_Access_Log_BaseLine

5.5 Controlled SQL Injection Test

A controlled SQL Injection test was performed against DVWA at Low security. The test demonstrated the impact of insufficient input handling in an intentionally vulnerable application.

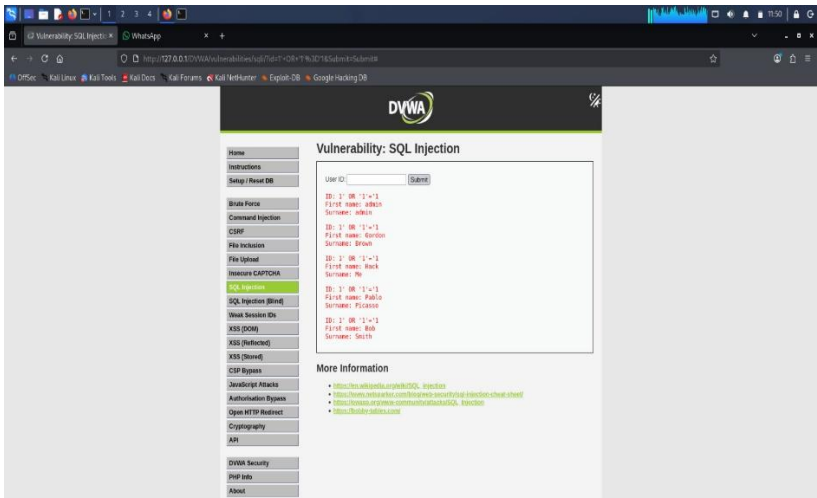


Figure 6.SQL_Injection_Test

Risk: In a real application, SQL Injection could potentially allow manipulation of database queries and unauthorized access or modification of data, depending on application design and database privileges.

5.6 Finding Summary

ID	Finding	Severity	Primary Mitigation
F-01	SQL Injection in intentionally vulnerable DVWA configuration	High	Prepared/parameterized queries; input validation; least privilege
F-02	Weak security configuration enabled for training	High (lab)	Secure production configuration and hardening
F-03	Web events require active monitoring	Medium	Centralized logging, monitoring and alerting

6. Incident Response Simulation

6.1 Scenario

A harmless request containing the parameter incident_response was generated against the local DVWA application to simulate suspicious web activity without causing damage or deploying malicious code.

6.2 Detection

The Apache access log was monitored and the new request was identified, demonstrating detection from available security telemetry.

[illegible]

Figure 7.Incident Detected In Log

6.3 Analysis

The Apache log was searched for the incident_response event and the request was reviewed to confirm the affected resource and recorded activity.

Evidence: Incident Analysis

6.4 Containment

A temporary iptables INPUT rule was applied to block the simulated local source address. This demonstrated a network-level containment action.

[illegible]

Figure 8. Incident Containment Firewall

6.5 Eradication

The temporary containment rule was removed after the simulation. No malware, persistence mechanism, or real compromise was introduced.

6.6 Recovery

After the temporary firewall rule was removed, DVWA was accessed again through the browser. The Home page loaded normally, confirming service recovery.

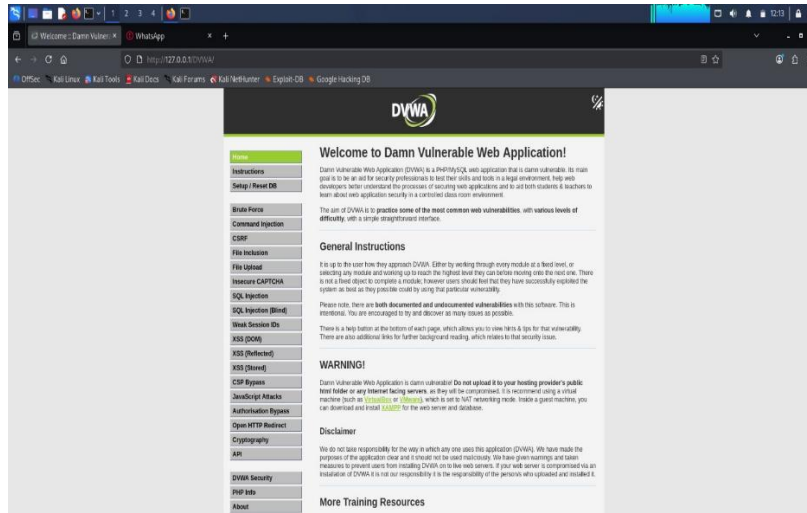


Figure 9.Incident_Recovery_Verification

6.7 Lifecycle Summary

Stage	Action	Evidence
Detect	Identified simulated request in Apache log	Incident_Detected_In_Log
Analyze	Searched and reviewed matching log entry	Incident_Analysis
Contain	Applied temporary iptables rule	Incident_Containment_Firewall
Eradicate	Removed temporary rule; no persistence present	Documentation
Recover	Verified DVWA normal operation	Incident_Recovery_Verification

7. Findings and Mitigations

7.1 SQL Injection

- Use prepared statements and parameterized SQL.
- Validate input using allow-list approaches where practical.
- Never concatenate untrusted input directly into SQL statements.
- Use least-privilege database accounts.
- Do not expose detailed database errors.
- Perform regular secure-code reviews and application security testing.

7.2 Web Application Hardening

- Use secure configurations rather than intentionally vulnerable settings.
- Keep Apache, PHP and application components patched.
- Apply appropriate security headers.
- Use secure authentication and session controls.
- Consider a Web Application Firewall as a defense-in-depth control.

7.3 Logging and Monitoring

- Centralize important web and application logs.
- Alert on suspicious requests and repeated attack patterns.
- Protect log integrity and define retention periods.
- Correlate web logs with other security telemetry where available.

7.4 Incident Response

- Maintain a documented incident-response procedure.
- Preserve relevant evidence before destructive changes.
- Document containment actions.
- Verify service health after containment and recovery.
- Conduct post-incident reviews and improve controls.

8. Post-Incident Report

8.1 Classification

Simulated Web Application Security Incident — controlled laboratory exercise.

8.2 Impact

No production system, external service, real user account, or real customer data was affected. The exercise was restricted to the local DVWA training environment.

8.3 Root Cause / Contributing Factor

The primary contributing factor was the intentionally insecure DVWA configuration at Low security. DVWA is designed for security training and contains known vulnerabilities.

8.4 Response Effectiveness

The event was successfully detected from Apache logs, analyzed, temporarily contained, cleared by removing the temporary rule, and followed by successful recovery verification.

8.5 Lessons Learned

- Web-server logs can provide useful first-line evidence.
- Detection should be followed by analysis before containment.
- Firewall controls can support containment when applied carefully.
- Recovery verification is necessary after containment changes.
- Security testing and incident response should be treated as connected processes.

9. Evidence Register

No.	Evidence	Purpose
01	DVWA_Security_Level_Low	DVWA security configuration
02	Nmap_DVWA	Reconnaissance/service scan
03	HTTP-Headers	HTTP response inspection
04	Apache_Access_Log_BaseLine	Baseline log review
05	SQL_Injection_Test	Controlled SQL Injection demonstration
06	Incident_Detected_In_Log	Incident detection
07	Incident_Analysis	Incident analysis
08	Incident_Containment_Firewall	Firewall containment
09	Incident_Recovery_Verification	Recovery verification

10. Conclusion

This capstone project successfully applied practical cybersecurity skills to a controlled DVWA environment and connected offensive security testing with defensive incident response. The work covered reconnaissance, service identification, HTTP analysis, controlled SQL Injection testing, evidence collection, log-based detection, analysis, containment, eradication, and recovery verification.

The project demonstrates that effective cybersecurity requires more than identifying vulnerabilities: it also requires reliable logging, evidence handling, appropriate containment, recovery validation, mitigation planning, and post-incident learning.